



BLUE BITS

البرمجة التفرعية

نظري (3)

ملف داعم

2026/2025

السنة الرابعة



مكتبة شهاب
كشياء الذائفة - يفرق بينك وبين الناس

[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

www.bluebitsteam.com



MPI: Message Passing Interface

- مكتبة تبادل الرسائل *Message Passing Library* معيار تم تطويره من قبل مجموعة من الأكاديميين والشركاء الصناعيين لتعزيز الاستخدام على نطاق أوسع والقدرة على النقل.
- *MPI* تملك عدد كبير من الروتينيات (أكثر من 120).
- جميع روتينيات *MPI* تبدأ بالبادئة *MPI_* والحرف بعدها حرف كبير.

MPI: Process Creation and Execution

- إن إنشاء العمليات بشكل ستاتيكي مدعوم فقط في ال *version1* من مكتبة *MPI* حيث يجب تعريف كل العمليات قبل التنفيذ وبدء تشغيلها معاً.
- في ال *version2* من مكتبة ال *MPI* تم اصدار ميزة متقدمة لإنشاء العمليات ديناميكياً؛ ويحتوي هذا الاصدار روتين الاستدعاء (*MPI_Comm_spawn()*).
- ومع ذلك قد لا يفضل استخدامه لأنه يزيد الحمل *overhead* الناتج عن إنشاء العمليات ديناميكياً.
- كما يمكن استخدام نموذج الحوسبة *SPMD* والذي فيه يكتب برنامج واحد ويتم تنفيذه على عدة معالجات
- نشغل برنامج ال *MPI* القابل للتنفيذ (بعد ال *debugging* يعني) على ال *commandline*
- على سبيل المثال، يمكن تشغيل نفس الملف التنفيذي على أربعة معالجات منفصلة في الوقت نفسه باستخدام *mpirun prog1 - n4*
- عملية ال *mapping* بين العمليات والمعالجات غير معرفة في معيار *MPI*
- قد تكون متاحة (ال *mapping*) عبر ال *commandline* او باستخدام ملف يحوي اسماء الملفات التنفيذية والمعالجات المحددة لتشغيل كل ملف تنفيذي.
- قبل أي استدعاء لتوابع *MPI* يجب تهيئة الكود باستخدام (*MPI_Init()*) وبعد جميع الاستدعاءات يجب ان ينتهي الكود باستخدام (*MPI_Finalize()*)

```
1 main(int argc, char * argv[])
2 {
3     MPI_Init(&argc, &argv);
4     ...
5     MPI_Finalize();
6 }
```

Communicators حوض الإتصال:

- في البداية تم تسجيل جميع العمليات في عالم يسمى *MPI_COMM_WORLD* وتم منح كل عملية مرتبة فريدة *Unique Rank* وهي رقم يتراوح من 0 وحتى $P - 1$ من أجل P من العمليات.
- *MPI_COMM_WORLD* هو حوض اتصال يحدد نطاق عملية اتصال.
- العمليات لها رتب مرتبطة بالمتصلين.
- يمكن إنشاء أحواض اتصال أخرى لمجموعة من العمليات.

MPI Solution: "Communicators"

- يعرف مجال الاتصال بأنه مجموعة من العمليات المسموح لها بالتواصل فيما بينها
- يمكن فصل مجالات الاتصال الخاصة بالمكتبات عن مجالات الاتصال الخاصة ببرنامج المستخدم



- تُستخدم (أي مجتمعات الاتصال) في جميع اتصالات نقل الرسائل *MPI* سواءً كانت *Point to Point* أو *Collective*

Default Communicator: *MPI_COMM_WORLD*

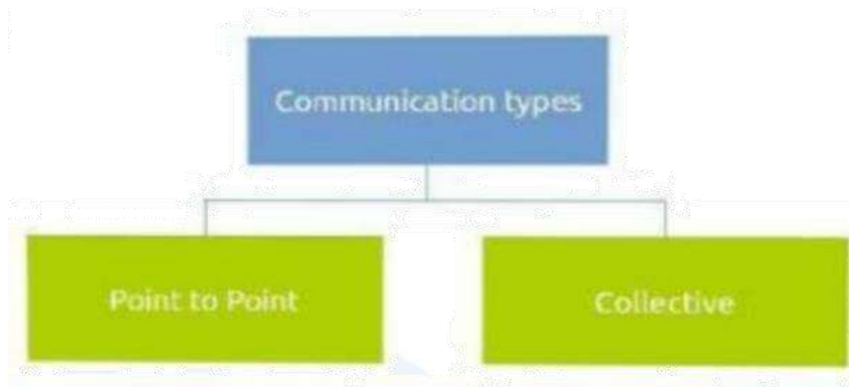
- يُستخدم كجهاز اتصال أول لجميع العمليات الموجودة في التطبيق.
- توجد مجموعة من إجراءات *MPI* لتشكيل أجهزة الاتصال.
- العمليات تملك رتبة *Rank* في جهاز الاتصال.

MPI data type	C data type
MPI_CHAR	Signed char
MPI_SHORT	Signed short int
MPI_INT	Signed int
MPI_LONG	Signed long int
MPI_UNSIGNED_CHAR	Unsigned char
MPI_UNSIGNED_SHORT	Unsigned short int
MPI_UNSIGNED_INT	Unsigned int
MPI_UNSIGNED_LONG	Unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	Long double
MPI_BYTE	
MPI_PACKED	



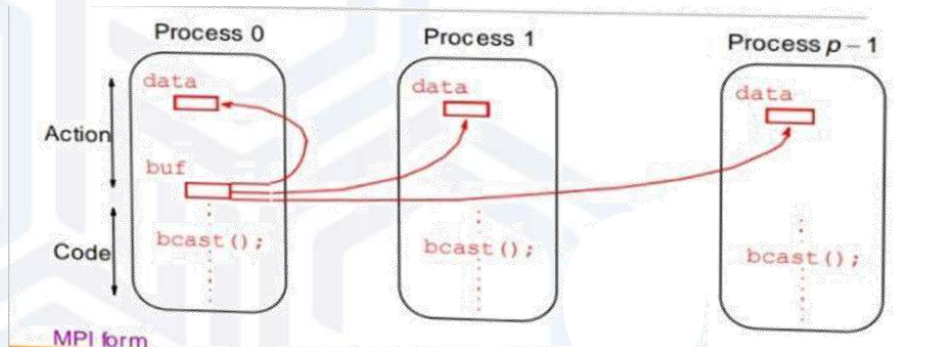
"Group" Message Passing Routines

- تحتوي على روتينيات ترسل رسائل إلى مجموعة من العمليات أو تستقبل رسائل من مجموعة من العمليات وتتميز بكفاءة أعلى من الروتينيات المنفصلة *Point to Point*



Broadcast

إرسال نفس الرسالة إلى جميع العمليات المعنية بالشبكة



ملاحظة:

تعد *buf* الحجرة التي تحتوي الـ *data* المخزنة عند الـ *process* المُرسلة

MPI_Bcast

MPI_Bcast(buf, count, datatype, source, comm)

عنوان مصفوفة الـ *data* المُرسلة

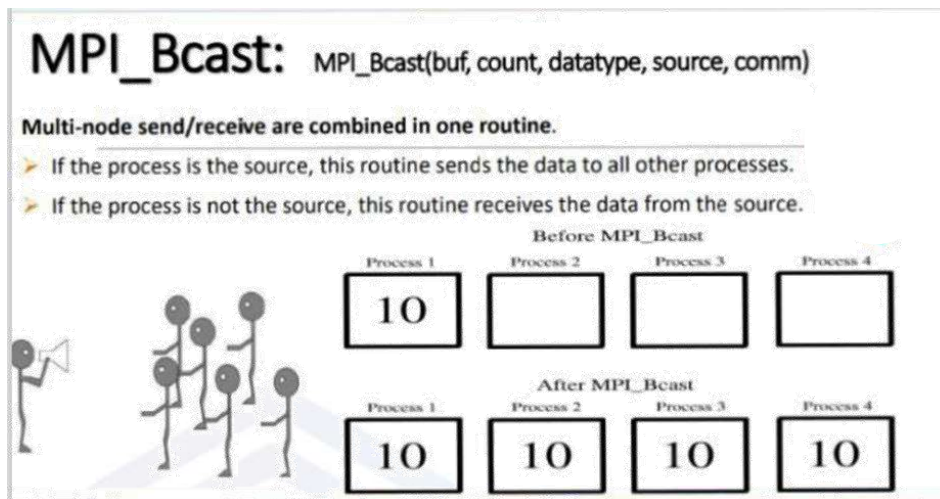
عدد القيم التي نريد إرسالها

نوع القيم التي نريد إرسالها

المصدر

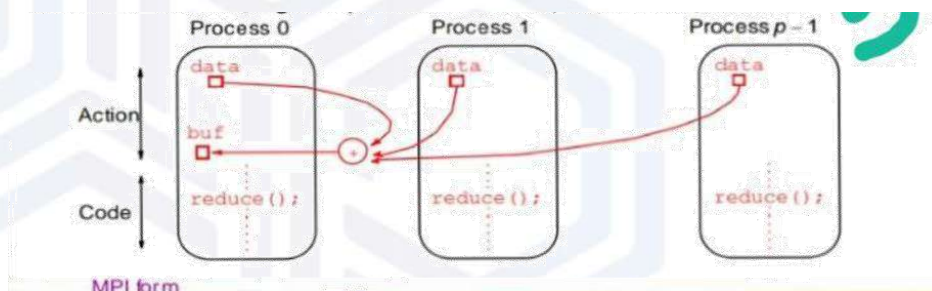
Communicator

- إذا كانت العملية هي المصدر يرسل هذا الروتين البيانات إلى جميع العمليات الأخرى
- إذا كانت العملية ليست هي المصدر يتلقى هذا الروتين البيانات من المصدر



Reduce

- عملية التجميع مع عملية حسابية/منطقية محددة
- مثال: يمكن تجميع القيم ثم جمعها بواسطة الجذر
- يمكن إجراء عملية حسابية/منطقية أخرى بواسطة الجذر

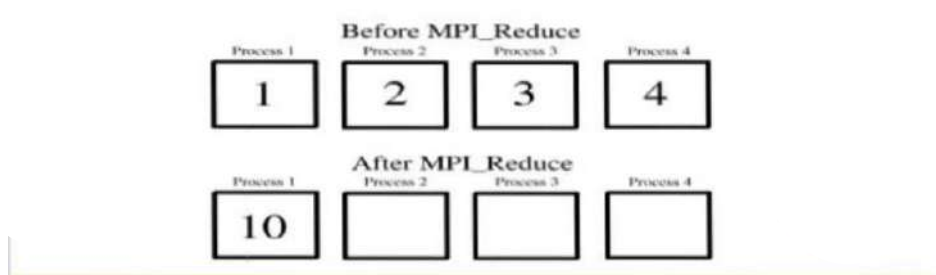


*MPI_Reduce*

MPI_Reduce(sendbuf, recvbuf, count, datatype, operation, dest, comm)

الوجهة ← العملية سواء كانت حسابية أو منطقية ← عناوين للداتا المرسل والمستقبل

- إذا كانت العملية ليست الوجهة إذا أرسل البيانات
- إذا كانت العملية هي الوجهة إذا تلقى البيانات, نفذ العملية و ضع النتيجة في *recvbuf*



مثال عن *Broadcast / Reduce*:

1. مطلوب جمع 1000 عنصر بحيث يقوم كل معالج بجمع عدد محدد من العناصر فقط مثلاً $P = 10$ و $N = 1000$ كل معالجة لتشتغل 100
- بداية عمل المعالجة

$$first = 1000 \setminus 10 * 0 + 1 = 1$$

- نهاية عمل المعالجة

$$last = 1000 \setminus 10 * (0 + 1) = 100$$

2. كل معالجة تبدأ من *first* وتنتهي عند *last* وتجمع 100 عدد جانب شرط ال *if* وال *sum*
3. عند $sum += i$ صار كل *process* عندها مجموعة وبالتالي بتابع ال *reduce* حتى أجمع كل هالمجموعات وأعمل عليها عملية حسابية وأخزنها في *process* معينة وكل معالجة تملك سرعة مختلفة عن الأخرى

```

1  if (myid == 0)
2  N = atoi(argv[1]);
3  MPI_Bcast(&N, 1, MPI_INT, 0, MPI_COMM_WORLD);
4  int first = N / numproc * myid + 1;
5  int last = N / numproc * (myid + 1);
6  for (i = first; i <= last; ++i)
7  sum += i;
8  MPI_Reduce(&sum, &result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
9  if (myid == 0)
10 printf("Sum = %d\n", result);

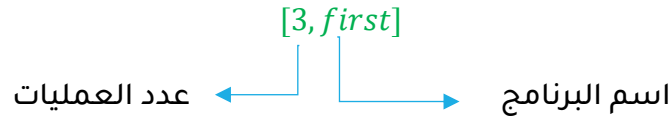
```



`main(int argc,* argv[])`

Argc: عبارة عن متحول يحتوي على البرامترات التي يتم تمريرها في *cmd*

Argv: عبارة عن مصفوفة تحوي على عناصر المتحولات



Atoi تحول *string* الى *int*

first: $N / \text{numproc} * (\text{myid} + 1)$ تابع

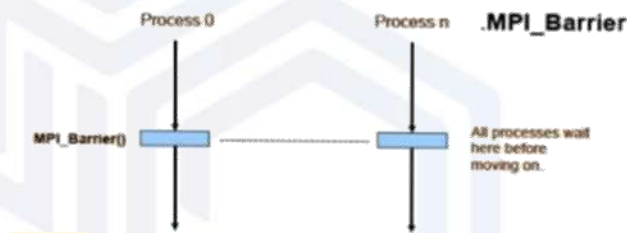
last: $N / \text{numproc} * (\text{myid} + 1)$ تابع

Collective Computation

MPI name	Operation
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Summation
MPI_PROD	Product
MPI_LAND	Logical AND
MPI_LOR	Logical OR
MPI_LXOR	Logical XOR

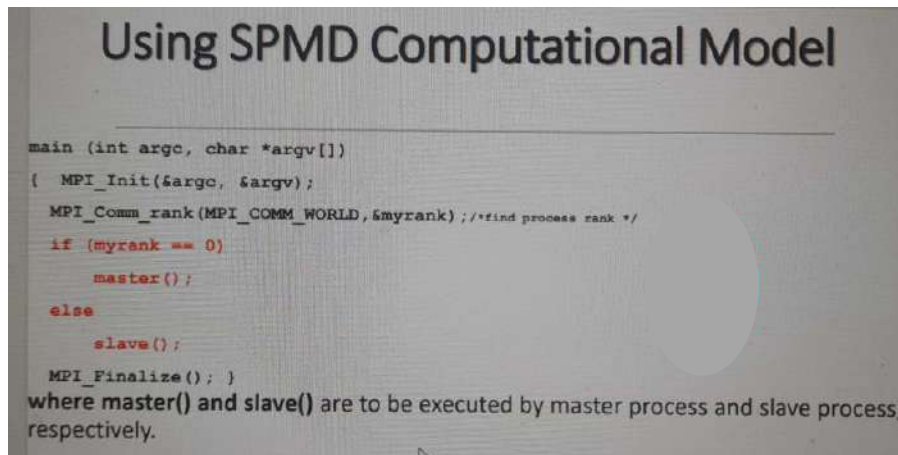
MPI_Barrier

طالما أن المهمات *processes* تعمل بشكل تفرعي و كل مهمة يمكن أن يكون زمن تنفيذها مختلف عن أزمته تنفيذ بقية المهمات تبعاً للمعالج الذي تعمل عليه، فمن الممكن أن تضطر في بعض الأحيان إلى المزامنة بين المهمات لتنفيذ عملية معينة، تقدم لنا *MPI* تعليمات تنفيذ المزامنة بين المهمات سنتعرف الآن على تعليمة *MPI_Barrier*



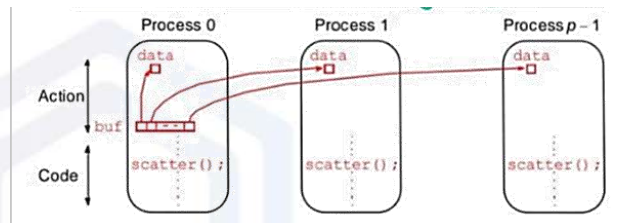
Barrier

- كما هو الحال في جميع أنظمة تمرير الرسائل، يوفر **MPI** وسيلة لمزامنة العمليات عن طريق إيقاف كل عملية حتى تصل جميعها إلى استدعاء **Barrier** محدد
- **MPI_Barrier(comm)**
- عندما تصل أول عملية إلى هذا الاستدعاء تقوم بالانتظار حتى تستدعي جميع العمليات هذا الإجراء قبل المتابعة
- عندما تقوم مهمة ما (عملية) بتنفيذ عملية **MPI_Barrier** فإنها تقوم بعمل إيقاف **block** ولا تتابع تنفيذ باقي البرنامج ريثما تقوم جميع المهمات باستدعاء وتنفيذ هذه العملية.

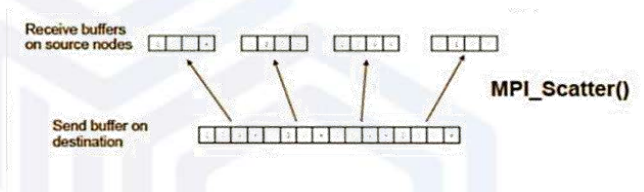


Scatter

إرسال كل عنصر من عناصر المصفوفة في العملية الجذرية إلى عملية منفصلة



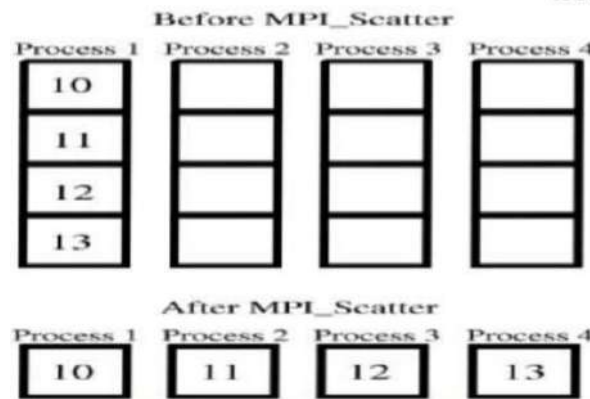
MPI_Scatter(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, root, comm)



MPI_Scatter

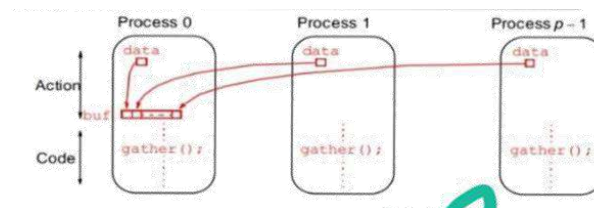
- يمكن تقسيم بارامترات هذه العملية إلى قسمين:
 - قسم يهم المرسل (المهمة الجذر) والذي يتمثل في أول ثلاث بارامترات.
 - قسم يهم المستقبل **recv_buffer - recv_count - datatype**

- في ذاكرة المهمة المستقبلية للبيانات يجب أن يكون المتحول `send_buffer` معرف ولكن يمكن أن تكون قيمته `NULL` حيث قيمته الحقيقية موجودة فقط في ذاكرة المهمة الجذر.
- عدد العناصر `count` المرسله يجب أن يكون مساو لعدد العناصر المستقبلية `send_count = recv_count`



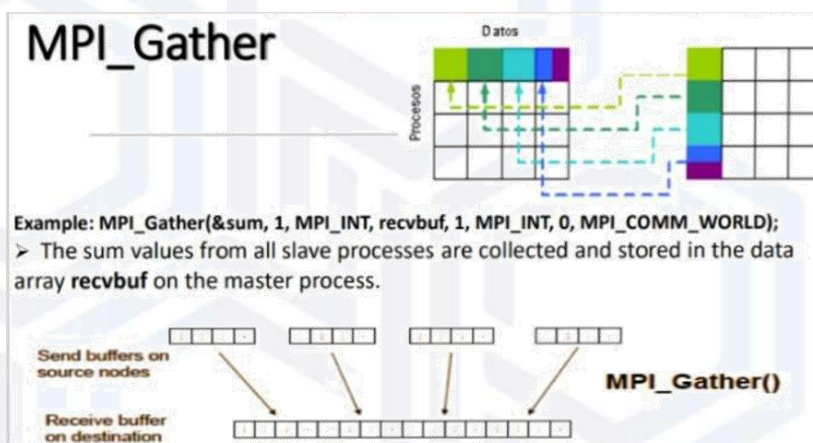
Gather

- عملية واحدة تجمع قيماً فردية من مجموعة عمليات
- عملية معاكسة لـ `Scatter`



- `MPI_Gather()` تقوم بالتجميع من كافة العمليات بما في ذلك الجذر.
- بدلاً من جمع النتائج باستخدام `MPI_Reduce` يمكننا جمع المجاميع الجزئية الفردية باستخدام `MPI_Gather()`

`MPI_Gather(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, root, comm)`





مثلاً ليكن لدي 4 مصفوفات كل مصفوفة فيها 10 عناصر لما بدي أجمعن بصير بدي 40 عنصر لذلك منجز 40 عنصر بال *buf*

مثال عن *Gather*:

تابع بلغة C من أجل تخزين حجرة ذاكرة *malloc*

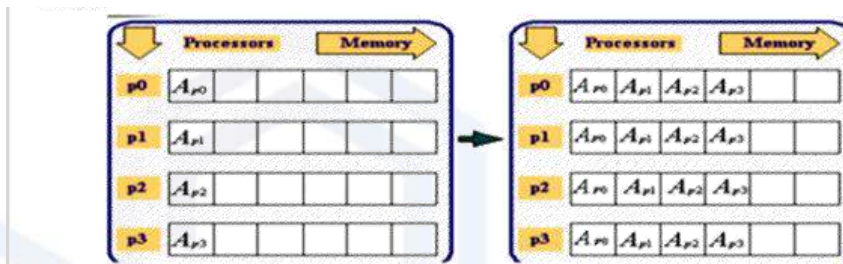
```
11  int data[10]; /* العمليات من جمعها سيتم التي البيانات */
12  MPI_Comm_rank(MPI_COMM_WORLD, &myrank); /* الرتبة تحديد */
13  if (myrank == 0) {
14      MPI_Comm_size(MPI_COMM_WORLD, &grp_size); /* المجموعة حجم تحديد */
15      buf = (int *)malloc(grp_size * 10 * sizeof(int)); /* الذاكرة تخصيص */
16  }
17  MPI_Gather(data, 10, MPI_INT, buf, grp_size * 10, MPI_INT, 0, MPI_COMM_WORLD);
```

MPI_ALLGATHER

هذا التابع يقوم بتجميع كل البيانات لدى كل العمليات، يستقبل من قبل الجميع ولا يقتصر على ال *root*

- آلية الاستقبال والتخزين ليست عشوائية، وإنما تتم بشكل تسلسلي.
- عند استدعاء التابع لدينا التالي:

```
Int MPI_ALLGATHER(void * sendbuf, int sendcount, MPI_Datatype sendtype, void
* recvbuf, int recvcount, MPI_Datatype recvtype, MPI_Comm comm)
```

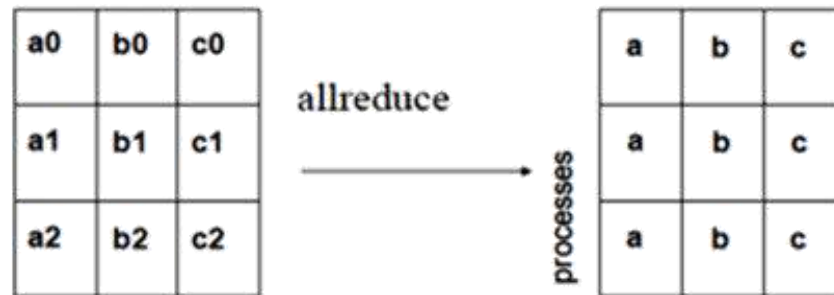


```
18  MPI_ALLGATHER(void * sendBuf, int sendCount, MPI_dataType sendType,
19  void * recvBuf, int recvCount, MPI_datType recvType, MPI_Comm Comm);
```

MPI_ALLREDUCE

تقدم جميع العمليات باستقبال نفس القيم وتطبيق نفس العملية وبالتالي كافة العمليات عندها نفس القيم ضمن نفس الترتيب بعد تطبيق العملية عليها.

```
Int MPI_ALLREDUCE(void * sendbuf, void
* recvbuf, int count, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```



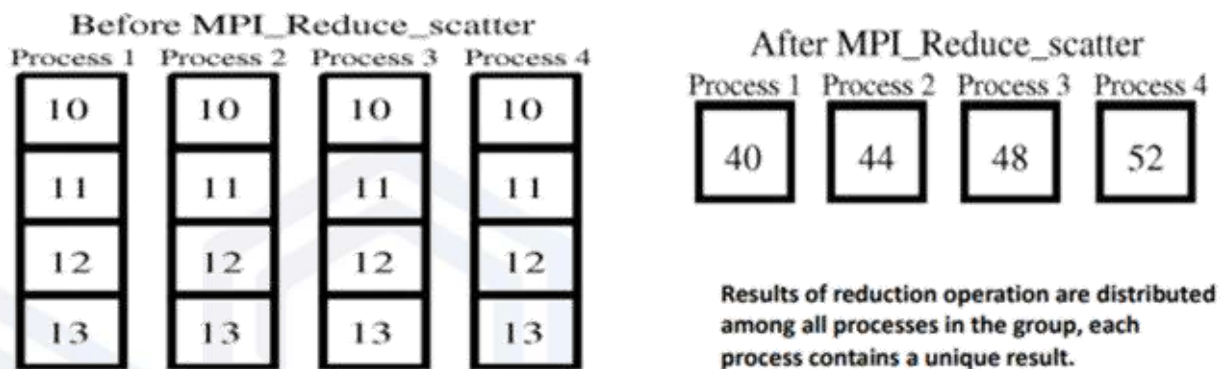
MPI_REDUCE_SCATTER

يعمل هذا التابع على مرحلتين:

أولاً: نقوم بتقطيع وإرسال القيم إلى جميع ال *Processes* أي تنفيذ *Scatter*

ثانياً: بعد إرسال القيم يقوم *reduce* بإجراء عملية حسابية/منطقية والتخزين.

- يتم توزيع نتائج عملية الاختزال (سواء حسابية أو منطقية) على جميع العمليات في المجموعة و كل عملية تحتوي على نتيجة فريدة.



```
int MPI_Reduce_scatter(void *sendbuf, void *recvbuf, int  
*recvcounts, MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```



ملخص المحاضرة الثالثة:

Message tag

للتمييز بين أنواع الرسائل المختلفة

مكتبة تبادل الرسائل *MPI*

- جميع روتينيات *MPI* تبدأ بالبداية *MPI_* والحرف بعدها كبير
- قبل أي استدعاء *MPI* يجب استدعاء تابع التهيئة *MPI_Init(&argc, &argv)*
- ينتهي الكود دوماً باستخدام *MPI_Finalize()*
- *MPI_COMM_WORLD* حوض اتصال يحوي جميع العمليات
- *Rank* مرتبة فريدة يتم منحها إلى كل عملية
- تابع يرسل نفس الرسالة إلى جميع العمليات
- *MPI_Bcast(buf, count, datatype, source, comm)*
- تابع يقوم بعملية تجميع للقيم مع عملية حسابية/منطقية
- *MPI_Reduce(sendbuf, recvbuf, count, datatype, operation, dest, comm)*
- تابع لمزامنة العمليات *MPI_Barrier(comm)*
- إرسال كل عنصر من عناصر المصفوفة إلى عملية منفصلة
- *MPI_Scatter(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, root, comm)*
- عملية واحدة تجمع قيماً فردية من مجموعة عمليات *MPI_Gather()*

2026/2025

السنة الرابعة



أعضاء الفريق

الفريق التقني

ظلال بابا	إيناس بيلوني
عائشة قلّه	تسنيم طالب
عائشة مُصطفى	حلا إبراهيم
قمر حَمَد	دلال تسقيه
محمد يمان جلال	ديانا صَبَّاغ
محمود غزال	زهير حَقّامي
وفاء حَقّامي	شكرية سُلطان
يوسف نِيّال	شهد كَرّوم

الفريق العلمي

عائشة العُمر	إبراهيم هبرة
العلّوش	إسراء حاج حمدي
عمر قَطّان	بتول لَبّان
فاطمة الحسين	بشرى كوكش
قاسم يحيى	بنيامين حكمان
كريست ياغجيان	تسنيم حوكان
محمد منير	تسنيم نور الدّين
مديحة جَمّال	جميل هلال
ميرا فضلي	جودي جلال
نجم الدّين درويش	خديجة بيازيد
هبة الله كتلو	رشا معراوي
وداد قلعيّة	سدرة مبيض
ياسر صَبّاغ	

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

